

BCD II: Business Components Development II

Comprehensive Lecture Notes & Practical Developer Guide — Enterprise Java (J2EE / Jakarta EE)

MODULE OVERVIEW

Course Module: BCD II (Business Components Development II)

Target Framework: Enterprise JavaBeans (EJB 3.x), Jakarta EE / Java EE Architecture

Core Topics Covered: Transaction Management, Timer Service, Web Services (JAX-WS & JAX-RS), Interceptors, Enterprise Security, EJB Exception Handling, Packaging & Deployment Strategy

1. Transaction Management in EJB

1.1 Core Concepts and Principles

Transaction management is a foundational pillar of enterprise application development. In J2EE/Jakarta EE, Enterprise JavaBeans (EJB) provide robust support for ensuring system state integrity across distributed operations through the ACID properties:

- **Atomicity:** Operations must complete entirely or not at all (all-or-nothing execution).
- **Consistency:** Transactions transition the system from one valid state to another, maintaining invariants and integrity constraints.
- **Isolation:** Concurrent transactions execute independently without interfering with each other's uncommitted state.
- **Durability:** Once committed, transaction results are permanently saved in persistent storage, surviving system crashes.

EJB supports two main paradigms for transaction management: Container-Managed Transactions (CMT) and Bean-Managed Transactions (BMT).

1.2 Container-Managed Transactions (CMT)

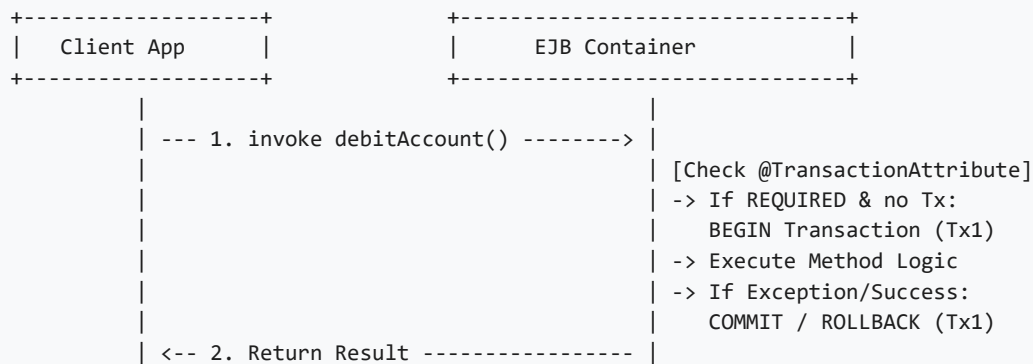
In CMT, the EJB container declaratively handles transaction boundaries (begin, commit, rollback) based on annotations or XML deployment descriptors. This abstracts low-level transaction boundary management from business logic.

CMT provides six transaction attributes defined in `javax.ejb.TransactionAttributeType`:

- **REQUIRED (Default):** Container invokes the method within an active transaction. If none exists, a new transaction is started.
- **REQUIRES_NEW:** Container always starts a new transaction. If an active transaction exists, it is suspended during method execution.

- **SUPPORTS:** Container invokes the method within the client's transaction context. If no transaction exists, the method executes non-transactionally.
- **MANDATORY:** Container requires a client transaction to exist. If none exists, `javax.ejb.EJBTransactionRequiredException` is thrown.
- **NOT_SUPPORTED:** Container executes the method non-transactionally. If a transaction exists, it is suspended during method execution.
- **NEVER:** Container verifies no transaction exists. If a transaction is active, `javax.ejb.EJBException` / `RemoteException` is thrown.

DIAGRAM / ARCHITECTURE FLOW: CMT Transaction Propagation Flow



Description & Workflow: Illustrating how the container automatically intercepts client calls, evaluates transaction attributes, opens database connections/transactions, and issues commit/rollback commands based on execution outcome.

1.3 Bean-Managed Transactions (BMT)

BMT gives developers explicit programatic control over transaction boundaries using the `javax.transaction.UserTransaction` interface. BMT is typically used when transactions must span multiple client user interactions or require dynamic, programmatic rollback control.

1.4 Java Code Implementation

```
// 1. Container-Managed Transaction Example
package com.jiat.bcd2.ejb;

import javax.ejb.Stateless;
import javax.ejb.TransactionManagement;
import javax.ejb.TransactionManagementType;
import javax.ejb.TransactionAttribute;
import javax.ejb.TransactionAttributeType;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
```

```

@Stateless
@TransactionManagement(TransactionManagementType.CONTAINER) // Default
public class AccountServiceBean {

    @PersistenceContext
    private EntityManager em;

    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void transferFunds(Long sourceId, Long targetId, double amount) {
        Account source = em.find(Account.class, sourceId);
        Account target = em.find(Account.class, targetId);

        source.debit(amount);
        target.credit(amount);

        // Explicit rollback trigger for business rule failure
        if (source.getBalance() < 0) {
            // Container will rollback transaction upon method exit
            throw new IllegalArgumentException("Insufficient funds for transfer");
        }
    }

    @TransactionAttribute(TransactionAttributeType.REQUIRES_NEW)
    public void logAuditTrail(String action) {
        AuditLog log = new AuditLog(action);
        em.persist(log); // Always committed independently of caller's transaction
    }
}

```

```

// 2. Bean-Managed Transaction Example
package com.jiat.bcd2.ejb;

```

```

import javax.annotation.Resource;
import javax.ejb.Stateless;
import javax.ejb.TransactionManagement;
import javax.ejb.TransactionManagementType;
import javax.transaction.UserTransaction;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;

```

```

@Stateless
@TransactionManagement(TransactionManagementType.BEAN)
public class ProgrammaticTxBean {

    @Resource
    private UserTransaction utx;

    @PersistenceContext

```

```

private EntityManager em;

public void executeComplexProcess() {
    try {
        utx.begin(); // Start explicit transaction boundary

        em.createQuery("UPDATE Account a SET a.status =
'ACTIVE'").executeUpdate();

        // Programmatic decision point
        boolean systemCheck = performThirdPartyCheck();
        if (!systemCheck) {
            utx.rollback(); // Explicit rollback
            return;
        }

        utx.commit(); // Commit transaction
    } catch (Exception e) {
        try {
            utx.rollback();
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}

private boolean performThirdPartyCheck() { return true; }
}

```



BEST PRACTICES & COMMON PITFALLS: TRANSACTIONS

Never mix BMT and CMT within the same EJB bean class.

Avoid long-running operations inside BMT transactions (e.g., waiting for user input or external REST web calls) as it holds database connections and locks, causing pool exhaustion.

System exceptions (runtime exceptions like `NullPointerException`) automatically trigger CMT rollback, whereas checked exceptions do NOT trigger rollback unless marked with `@ApplicationException(rollback=true)`.

2. EJB Timer Service

2.1 Principles & Architecture

The EJB Timer Service is an enterprise scheduling facility integrated directly into the EJB container. It allows applications to schedule background batch processing, periodic data synchronization, notification triggers, and cleanup tasks.

Key characteristics of EJB Timers:

- **Persistence:** Timers are saved to persistent storage (database/journal) by default. If the server crashes or restarts, active timers are restored.
- **Transactional Support:** Timers operate within the transaction context of the method that created them or the timeout callback.
- **Declarative vs. Programmatic:** EJB 3.1 introduced declarative cron-like scheduling via annotations alongside programmatically managed timers.

2.2 Automatic (Declarative) Timers

Automatic timers are defined using the `@Schedule` or `@Schedules` annotations on bean methods. The container creates and initializes these timers automatically upon application deployment.

2.3 Programmatic Timers

Programmatic timers are dynamically created at runtime using the `javax.ejb.TimerService` API (injected via `@Resource`). They allow custom dynamic intervals, single-action timeouts, and metadata passing via serializable info objects.

2.4 Java Code Implementation

```
package com.jiat.bcd2.timer;

import javax.annotation.Resource;
import javax.ejb.Schedule;
import javax.ejb.Schedules;
import javax.ejb.Stateless;
import javax.ejb.Timeout;
import javax.ejb.Timer;
import javax.ejb.TimerConfig;
import javax.ejb.TimerService;
import java.io.Serializable;
import java.util.Date;

@Stateless
public class EnterpriseSchedulerBean {

    @Resource
    private TimerService timerService;

    // 1. Declarative Cron-based Automatic Timer (Runs every midnight)
    @Schedule(second = "0", minute = "0", hour = "0", dayOfWeek = "Mon-Fri",
persistent = true)
    public void executeMidnightBatchProcessing() {
        System.out.println("Running daily enterprise batch process: " + new Date());
    }

    // Multiple schedule rules on a single method
    @Schedules({
```

```

        @Schedule(hour = "6", minute = "0"),
        @Schedule(hour = "18", minute = "0")
    })
    public void performBiDailySync() {
        System.out.println("Synchronizing inventory data...");
    }

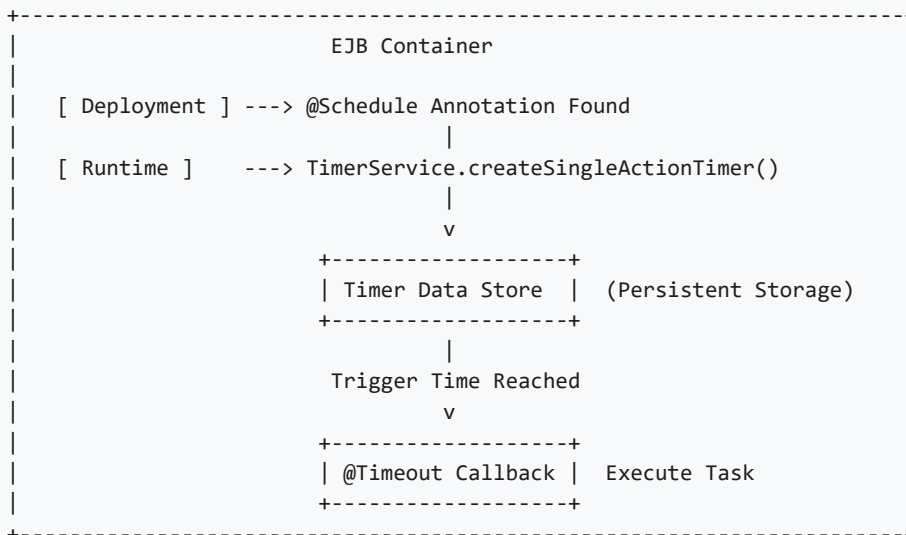
    // 2. Programmatic Timer Creation
    public void createDynamicTimer(long durationMillis, String taskInfo) {
        TimerConfig config = new TimerConfig();
        config.setInfo(taskInfo); // Attach custom payload metadata
        config.setPersistent(true);

        // Single-action timer triggered after specified delay
        timerService.createSingleActionTimer(durationMillis, config);
    }

    // Callback method for programmatic timer timeout
    @Timeout
    public void handleTimeout(Timer timer) {
        Serializable info = timer.getInfo();
        System.out.println("Programmatic Timer Fired! Task Metadata: " + info);
    }
}

```

DIAGRAM / ARCHITECTURE FLOW: EJB Timer Service Lifecycle



Description & Workflow: Lifecycle depicting timer registration (declarative or programmatic), persistence into the application server backing database, and scheduled invocation of the callback method.

BEST PRACTICES & COMMON PITFALLS: TIMERS

Ensure timer callback methods have a void return type and take either no parameters or a single parameter of type `javax.ejb.Timer`.

Do not use non-persistent timers for critical tasks if your system relies on recovery across cluster nodes or server restarts.

In clustered environments, ensure automatic timers are configured correctly so that they run on a single designated node rather than executing concurrently on every instance.

3. Web Services Integration (JAX-WS & JAX-RS)

3.1 Principles of Enterprise Web Services

Web services facilitate interoperable machine-to-machine interactions over a network. Enterprise JavaBeans can easily be exposed as remote web services to provide enterprise capabilities (transactions, security, pooling) to heterogeneous clients.

- **SOAP Web Services (JAX-WS):** Protocol: XML over HTTP/SOAP. Features: Strict WSDL contract definition, built-in WS-Security, WS-Addressing, WS-ReliableMessaging.
- **RESTful Web Services (JAX-RS):** Protocol: Lightweight HTTP (GET, POST, PUT, DELETE). Formats: JSON, XML, Plain Text. Ideal for mobile clients, single-page applications (SPAs), and RESTful microservices.

3.2 Java Code Implementation

```
// 1. SOAP Web Service using JAX-WS + Stateless EJB
package com.jiat.bcd2.ws;

import javax.ejb.Stateless;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.jws.WebService;

@Stateless
@WebService(
    name = "PaymentProcessorService",
    targetNamespace = "http://ws.bcd2.jiat.com/"
)
public class PaymentWebService {

    @WebMethod(operationName = "processPayment")
    public String processPayment(@WebParam(name = "accountNo") String accountNo,
                                @WebParam(name = "amount") double amount) {
        return "SUCCESS: Processed $" + amount + " for Account " + accountNo;
    }
}
```

```
// 2. RESTful Web Service using JAX-RS + EJB
package com.jiat.bcd2.ws;

import javax.ejb.Stateless;
import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;

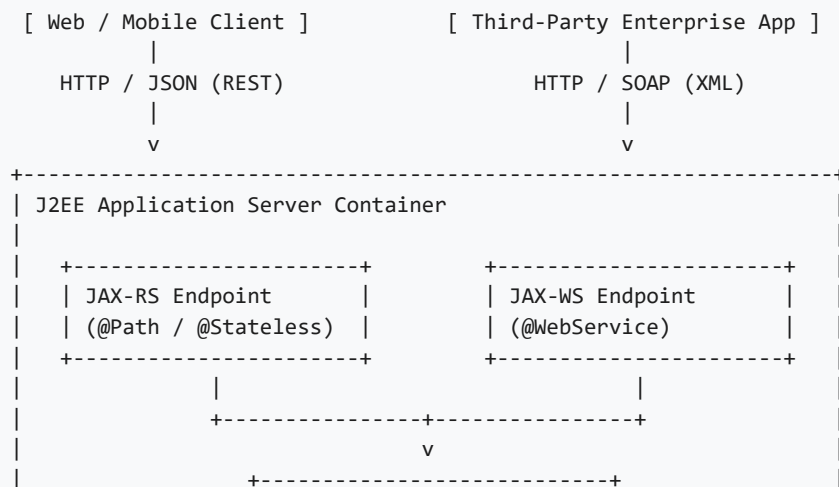
@Path("/customers")
@Stateless
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class CustomerRestService {

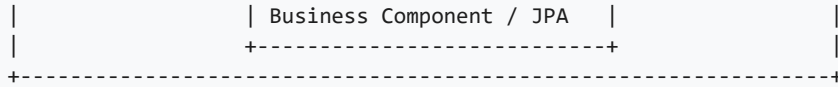
    @GET
    @Path("/{id}")
    public Response getCustomer(@PathParam("id") Long id) {
        Customer c = new Customer(id, "Lakshan", "lakshan@jiat.edu.lk");
        return Response.ok(c).build();
    }

    @POST
    public Response createCustomer(Customer customer) {
        // Business logic to persist customer
        return Response.status(Response.Status.CREATED)
            .entity(customer)
            .build();
    }
}

```

DIAGRAM / ARCHITECTURE FLOW: Web Services Architecture & EJB Integration





Description & Workflow: Integration of RESTful and SOAP endpoints implemented directly as Stateless Session Beans, leveraging enterprise runtime capabilities.

4. Interceptors

4.1 Concepts & Aspect-Oriented Programming (AOP)

Interceptors implement Aspect-Oriented Programming (AOP) concepts within the J2EE environment. They allow developers to separate cross-cutting concerns—such as logging, security checking, performance auditing, and validation—from business logic.

Interceptor Lifecycle Interception Points:

- **Around-Invoke Interceptors:** Intercepts calls to business methods (`@AroundInvoke`).
- **Lifecycle Callback Interceptors:** Intercepts bean lifecycle events like `@PostConstruct`, `@PreDestroy`, `@PostActivate`, and `@PrePassivate`.
- **Around-Timeout Interceptors:** Intercepts timeouts triggered by the EJB Timer Service (`@AroundTimeout`).

4.2 Java Code Implementation

```
package com.jiat.bcd2.interceptor;

import javax.interceptor.AroundInvoke;
import javax.interceptor.InvocationContext;
import javax.interceptor.Interceptors;
import javax.ejb.Stateless;
import java.util.logging.Logger;

// 1. Interceptor Class
public class PerformanceLoggingInterceptor {

    private static final Logger logger =
        Logger.getLogger(PerformanceLoggingInterceptor.class.getName());

    @AroundInvoke
    public Object logExecutionTime(InvocationContext context) throws Exception {
        String methodName = context.getMethod().getName();
        String className = context.getTarget().getClass().getName();
        long startTime = System.currentTimeMillis();

        logger.info("--> Entering Method: " + className + "." + methodName);
    }
}
```

```

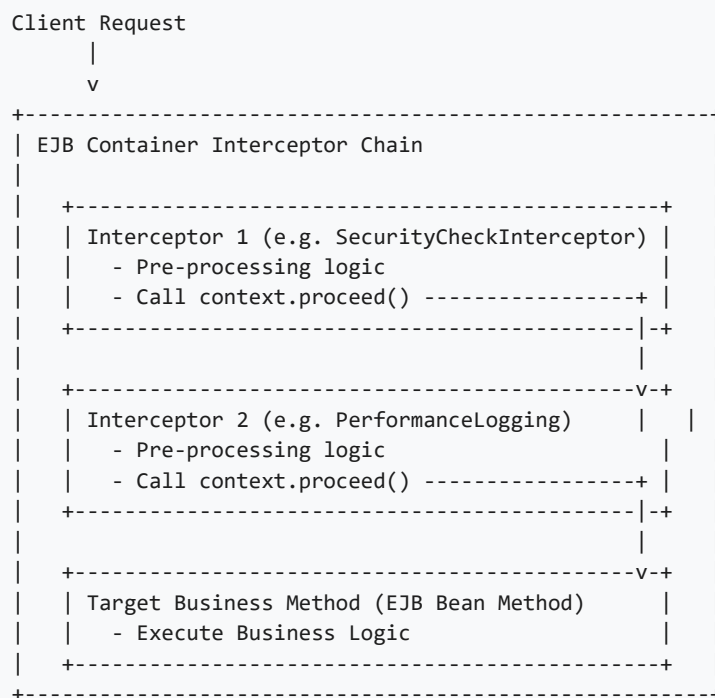
    try {
        // Proceed to the next interceptor in chain or actual EJB method
        Object result = context.proceed();
        return result;
    } finally {
        long duration = System.currentTimeMillis() - startTime;
        logger.info("<-- Exiting Method: " + methodName + " [Execution Time: " +
duration + " ms]");
    }
}
}

// 2. Applying Interceptor to EJB Bean
@Stateless
@Interceptors(PerformanceLoggingInterceptor.class) // Class-level binding
public class OrderProcessingBean {

    public void processOrder(Long orderId) {
        System.out.println("Processing business logic for order: " + orderId);
    }
}

```

DIAGRAM / ARCHITECTURE FLOW: Interceptor Chain Execution Model



Description & Workflow: Sequential flow through multiple chained interceptors using context.proceed() to advance processing down to the target bean method.

5. EJB Security

5.1 Security Architecture

J2EE security provides a robust framework based on Declarative and Programmatic Security models. The container handles authentication (verifying identity) and authorization (verifying permissions) using defined Security Realms, Users, and Roles.

- **Declarative Security:** Configured via annotations (@RolesAllowed, @PermitAll, @DenyAll, @RunAs) or web.xml / ejb-jar.xml descriptors without code modification.
- **Programmatic Security:** Implemented directly within business code using javax.ejb.SessionContext to perform fine-grained authorization checks.

5.2 Java Code Implementation

```
package com.jiat.bcd2.security;

import javax.annotation.Resource;
import javax.annotation.security.DeclareRoles;
import javax.annotation.security.DenyAll;
import javax.annotation.security.PermitAll;
import javax.annotation.security.RolesAllowed;
import javax.annotation.security.RunAs;
import javax.ejb.SessionContext;
import javax.ejb.Stateless;
import java.security.Principal;

@Stateless
@DeclareRoles({"ADMIN", "MANAGER", "USER"})
@RolesAllowed({"ADMIN", "MANAGER"}) // Class-level default role restriction
public class PayrollServiceBean {

    @Resource
    private SessionContext sessionContext;

    @RolesAllowed("ADMIN")
    public void approveExecutiveSalary(Long empId, double newSalary) {
        // Programmatic check using SessionContext
        Principal caller = sessionContext.getCallerPrincipal();
        System.out.println("User " + caller.getName() + " executing salary
adjustment.");

        if (sessionContext.isCallerInRole("ADMIN")) {
```

```

        System.out.println("Admin role verified programmatically.");
    }
}

@PermitAll
public String getPublicNotice() {
    return "Payroll system maintenance scheduled for Sunday.";
}

@DenyAll
public void RestrictedInternalMethod() {
    // Unusable by any remote caller
}
}

```



SECURITY BEST PRACTICES

Prefer Declarative Security (@RolesAllowed) over Programmatic Security whenever possible to enforce clean separation of concerns.

Use @RunAs("RoleName") with caution when calling downstream beans, as it elevates the caller identity privileges for internal component interactions.

6. EJB Exception Handling

6.1 Application vs. System Exceptions

EJB differentiates between Application Exceptions and System Exceptions to determine transaction outcomes and bean instance lifecycles.

- **Application Exceptions:** Business-logic exceptions (usually checked exceptions like `InsufficientFundsException`). They do NOT cause automatic transaction rollback unless explicitly declared with `@ApplicationException(rollback = true)`. The bean instance remains alive.
- **System Exceptions:** Unchecked runtime exceptions (e.g., `NullPointerException`, `EJBException`, `PersistenceException`). The container automatically rolls back the current transaction, destroys the bean instance, and wraps the exception in a `RemoteException/EJBException` for the client.

6.2 Exception Handling Matrix

Exception Category	Example	Tx Rollback Behavior	Bean Instance Lifecycle
Default Application Exception	Checked Exception (CustomException)	No Automatic Rollback	Preserved in Pool

Configured Application Exception	@ApplicationException(rollback=true)	Automatic Rollback	Preserved in Pool
----------------------------------	--------------------------------------	--------------------	-------------------

System Exception	NullPointerException / EJBException	Automatic Rollback	Destroyed by Container
------------------	-------------------------------------	--------------------	------------------------

6.3 Java Code Implementation

```
package com.jiat.bcd2.exception;

import javax.ejb.ApplicationException;
import javax.ejb.Stateless;

// Custom Application Exception configured to trigger Rollback
@ApplicationException(rollback = true)
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

@Stateless
public class BankingBean {

    public void withdraw(double amount, double balance) throws
    InsufficientFundsException {
        if (amount > balance) {
            // Throws application exception -> Container triggers rollback, keeps bean
            alive
            throw new InsufficientFundsException("Balance insufficient: $" + balance);
        }

        if (balance == 0) {
            // Unhandled RuntimeException -> System Exception -> Rollback & Destroy
            bean instance
            throw new NullPointerException("Balance state invalid");
        }
    }
}
```

7. Packaging and Deploying EJB Applications

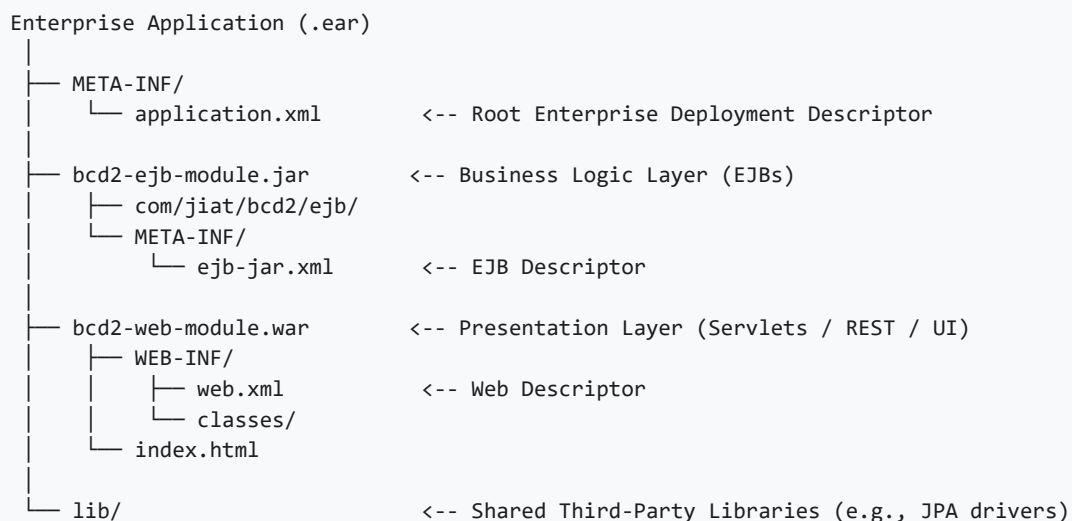
7.1 Enterprise Packaging Formats

J2EE applications follow strict modular packaging standard formats:

- **EJB JAR (Java Archive):** Contains EJB classes, interfaces, and optional META-INF/ejb-jar.xml deployment descriptor.
- **WAR (Web Application Archive):** Contains servlets, JSPs, REST endpoints, web assets (HTML/CSS/JS), WEB-INF/web.xml.
- **EAR (Enterprise Application Archive):** Top-level enterprise bundle containing EJB-JARs, WARs, library JARs, and META-INF/application.xml descriptor.

7.2 Deployment Descriptors & Structure

DIAGRAM / ARCHITECTURE FLOW: Standard Enterprise Archive (EAR) Directory Structure



Description & Workflow: Hierarchical layout of an Enterprise Archive (EAR) containing packaged EJB JARs, Web WAR modules, and shared dependencies.

7.3 Maven Deployment Descriptor Setup

```
<!-- Sample application.xml Deployment Descriptor -->
<application xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/application_7.xsd"
  version="7">
```

```
<display-name>BCD2-Enterprise-App</display-name>

<module>
  <ejb>bcd2-ejb-module.jar</ejb>
</module>

<module>
  <web>
    <web-uri>bcd2-web-module.war</web-uri>
    <context-root>/bcd2-app</context-root>
  </web>
</module>
</application>
```



DEPLOYMENT TIPS

In EJB 3.1+, EJB components can be directly packaged inside a WAR file under WEB-INF/classes, simplifying deployment for lightweight applications without requiring a full EAR structure.

Ensure deployment descriptors declare correct schema XML references matching the application server runtime version (e.g., WildFly, Payara, GlassFish, WebLogic).